MagicBox Installation Guide

Goal: Portainer reachable from the LAN at **https://magicbox.local:9443/**, with the `homelab` environment and the default stacks (RabbitMQ, PostgreSQL, Ollama + Open WebUI, Grafana + Loki + Alloy). MagicBox has no fixed IP: it is always reached as `magicbox.local` (mDNS via Avahi). `magicbox$` = on the Mini PC, `controller$` = on your workstation in this repository. Replace `<...>` with your values.

1. Manual Steps (Mini PC)

1.1 Before you start

- [ ] Back up anything on the internal disk (the OS install erases it).
- [ ] Have: USB stick (4 GB+), Ethernet cable to the LAN (recommended) or the Wi-Fi name and password, router admin access.
- [ ] Controller and Mini PC on the same subnet (not guest Wi-Fi).

1.2 Firmware (UEFI)

Setup key is usually `Del`, boot menu `F7` (check the boot screen).

- [ ] Apply a GMKtec firmware update for your model, if one exists.
- [ ] Boot mode: UEFI. Secure Boot: enabled.
- [ ] *Restore on AC Power Loss*: **Power On**.
- [ ] Save and exit.

1.3 Install Ubuntu Server 24.04 LTS

(Also supported: Ubuntu 26.04, Debian 12/13.)

- [ ] Download the ISO from `https://ubuntu.com/download/server`, verify its SHA-256, write it to the USB stick, boot from it.
- [ ] Network: wired or Wi-Fi, DHCP (no static IP). Skipped Wi-Fi: see 1.3.1.
- [ ] Storage: entire internal disk, **no** passphrase encryption (it blocks unattended reboots).
- [ ] Server name: **`magicbox`**. Create your admin user (`<admin-user>`).
- [ ] Tick **Install OpenSSH server**. Featured snaps: **none**.
- [ ] Reboot, remove the USB stick, log in.

1.3.1 Wi-Fi (only if skipped during install)

Replace the three `<...>` values (`ip -br link` shows the Wi-Fi interface, usually starting with `wl`) and run:

```
magicbox$ sudo netplan set --origin-hint 60-wifi 'wifis.<wifi-interface>={dhcp4: true,
access-points: {"<wifi-name>": {password: "<wifi-password>"}}}'
magicbox$ sudo netplan apply
```

The first command writes `/etc/netplan/60-wifi.yaml` (readable by root only). If the name or password contains `'`, `"` or `\`, edit that file instead.

1.4 Update and collect facts

```
magicbox$ sudo apt update && sudo apt full-upgrade -y && sudo reboot
magicbox$ hostname                          # must print magicbox
magicbox$ cat /etc/os-release; uname -m     # supported OS, x86_64
magicbox$ ip -br link                       # LAN interface name
magicbox$ ip -br -4 addr                    # LAN subnet (e.g. 192.168.1.x/24)
```

1.5 Avahi (magicbox.local)

```
magicbox$ sudo apt install -y avahi-daemon avahi-utils libnss-mdns
magicbox$ sudo systemctl enable --now avahi-daemon
magicbox$ avahi-resolve-host-name -4 magicbox.local
```

- [ ] The last command prints `magicbox.local` and an IPv4 address.
- [ ] From the controller: `ping -c 3 magicbox.local` answers. (Linux controller: `sudo apt install avahi-daemon libnss-mdns` first.)

1.6 SSH key

```
controller$ mkdir -m 700 -p ~/.ssh/magicbox
controller$ ssh-keygen -t ed25519 -f ~/.ssh/magicbox/id_ed25519 -C magicbox
controller$ ssh-copy-id -i ~/.ssh/magicbox/id_ed25519.pub <admin-user>@magicbox.local
controller$ ssh -4 -i ~/.ssh/magicbox/id_ed25519 <admin-user>@magicbox.local 'sudo -v && echo
sudo-ok'
```

- [ ] Dedicated key for MagicBox: private `~/.ssh/magicbox/id_ed25519` (never share it), public `~/.ssh/magicbox/id_ed25519.pub`.
- [ ] Last command prints `sudo-ok`.
- [ ] Optional, after key login works (keep a session open while testing):

```
magicbox$ echo 'PasswordAuthentication no' | sudo tee /etc/ssh/sshd_config.d/10-magicbox.conf
magicbox$ sudo sshd -t && sudo systemctl reload ssh
```

1.7 Values for the inventory

Value	Source	Yours
ansible_host	always magicbox.local	magicbox.local
ansible_user	admin user	_____
ansible_ssh_private_key_file	key from 1.6	~/.ssh/magicbox/id_ed25519
ansible_ssh_extra_args	keep (IPv4 only)	"-4"
ansible_become_exe	Ubuntu 26.04 only	sudo.ws
magicbox_lan_interface	ip -br link	_____
magicbox_lan_cidr	LAN subnet, e.g. 192.168.1.0/24	_____

2. Controller setup

- [ ] Install `uv` (`brew install uv`), `git`, `make`. Windows: use WSL2.

```
controller$ make setup
controller$ make check
```

3. Configuration

```
controller$ cp inventory/hosts.example.yml inventory/hosts.yml
controller$ make vault-edit          # creates the encrypted vault on first use
```

- [ ] In `inventory/hosts.yml`: set `ansible_user`, `magicbox_lan_interface`, `magicbox_lan_cidr` (1.7). Keep `ansible_host: magicbox.local` and `ansible_ssh_private_key_file: ~/.ssh/magicbox/id_ed25519`.
- [ ] Ubuntu 26.04: uncomment `ansible_become_exe: sudo.ws`.
- [ ] In `vault.yml` (see `inventory/vault.example.yml`): `vault_become_password` (sudo password of `ansible_user`) and these different values of 12+ characters: `vault_portainer_admin_password`, `vault_postgresql_password`, `vault_rabbitmq_password`, `vault_grafana_admin_password`, `vault_openwebui_admin_password`, `vault_jenkins_admin_password`, `vault_registry_password`; `vault_openwebui_secret_key` (`openssl rand -hex 32`), `vault_kong_ollama_api_key` (`openssl rand -hex 24`), `vault_rustfs_access_key` (`openssl rand -hex 12`), `vault_rustfs_secret_key` (`openssl rand -hex 24`). Leave `vault_jenkins_github_token` and `vault_jenkins_portainer_token` empty for now (4.1). Keep the vault password in your password manager.

```
controller$ make check
controller$ uv run ansible magicbox -m ansible.builtin.ping    # expect "pong"
```

4. Deployment

One idempotent command installs everything: host, Docker, Portainer, every stack (RabbitMQ, PostgreSQL, Ollama + Open WebUI, Grafana/Loki/Alloy, RustFS, registry, Jenkins, Kong) and the apps registry.

```
controller$ make deploy EXTRA="-e magicbox_confirm_destructive=true"   # first run
controller$ make deploy                                                # every later run
```

`make` asks only for the vault password; the sudo password comes from `vault_become_password`. The first run needs the confirmation flag because it enables the firewall. It pulls about 6 GB of images and builds the Jenkins images (allow 30+ minutes). The CA certificate is saved to `artifacts/magicbox-local-ca.crt`.

- [ ] The run ends with `failed=0`.
- [ ] A second `make deploy` reports `changed=0`.
- [ ] Open WebUI: `https://magicbox.local:8444/` opens; log in as `admin@magicbox.local` / `vault_openwebui_admin_password`; with a model pulled (`stacks_ollama_models`), a chat answers.
- [ ] If it stops waiting for containers (slow pulls): `make status`, then `make deploy` again (it resumes).

Run `make deploy` again after any change: vault values, `inventory/hosts.yml`, the apps registry, a new version of this repository.

4.1 CI/CD (Jenkins)

- [ ] `https://magicbox.local:8443/jenkins/` opens (same CA as Portainer); log in as `admin` / `vault_jenkins_admin_password`.
- [ ] [ ] *Manage Jenkins* > *Nodes*: `docker-agent` is online.

GitHub token (to read private repositories and show build status on commits):

- [ ] GitHub > Settings > Developer settings > Fine-grained tokens > Generate: only the application

repositories; permissions *Contents: Read*, *Commit statuses: Read and write*; set an expiry date.
- [ ] `make vault-edit`: `vault_jenkins_github_token: "<token>"`.
- [ ] In `inventory/hosts.yml` (vars): `stacks_jenkins_github_user: <github-user>`.

Portainer token (for pipelines that deploy stacks):

- [ ] Portainer > Administration > Users: add user `jenkins` (administrator).
- [ ] Log in as `jenkins` > My account > Access tokens > Add: copy the token.
- [ ] `make vault-edit`: `vault_jenkins_portainer_token: "<token>"`.

```
controller$ make deploy           # adds credentials "github" and "portainer-api-token"
```

- [ ] *Manage Jenkins* > *Credentials* lists `github`, `portainer-api-token` and `registry`.
- [ ] Repositories private; do not build pull requests from forks.

4.2 API gateway (Kong)

- [ ] `https://magicbox.local/` opens Portainer (redirect to `/portainer/`).
- [ ] `https://magicbox.local/grafana`, `/rabbitmq`, `/jenkins` (with or without the final `/`) open their login page.
- [ ] `https://magicbox.local/openwebui` and `/rustfs` redirect to their own ports; `/registry/` opens the registry UI.
- [ ] Ollama (API only, no UI) answers with the key, `401` without it:

```
controller$ curl -H "apikey: <vault_kong_ollama_api_key>"
https://magicbox.local/ollama/api/version
controller$ curl -s -o /dev/null -w '%{http_code}\n' https://magicbox.local/ollama/api/version
 # 401
```

Key	Where	Used by
vault_kong_ollama_api_key	vault	Header apikey: <key> (or ?apikey=<key>) on https://magicbox.local/ollama/

Rotate the key: change it with `make vault-edit`, run `make deploy`, update the clients.

4.3 Apps (Jenkins jobs, gateway routes)

Needs 4.1 (credentials `github` and `portainer-api-token`). New app: `manuals/how-to-create-new-app.pdf`.

```
controller$ cp inventory/apps.example.yml inventory/group_vars/magicbox/apps.yml   # if missing
```

- [ ] `inventory/group_vars/magicbox/apps.yml` (git-ignored): one entry per app with `name`, `port` (18000-18999), `repo` (GitHub HTTPS URL).
- [ ] The GitHub token (4.1) can read every `repo` listed.

```
controller$ make deploy
```

- [ ] Jenkins shows one job per app; new jobs start their first scan.
- [ ] Each app's `main` build runs Lint, Test, Build, Push, Deploy, Verify (green); `https://magicbox.local/<name>` opens.
- [ ] `make deploy` again: every app `deployed`, `changed=0`.

4.4 Object storage (RustFS)

- [ ] `https://magicbox.local:9101/rustfs/console/` opens (same CA as Portainer); log in with the access key and the secret key.
- [ ] Console: create a bucket, upload a file, download it.
- [ ] S3 client (endpoint `https://magicbox.local:9100`, path-style, region `us-east-1`, the MagicBox CA trusted):

```
controller$ AWS_ACCESS_KEY_ID=<access> AWS_SECRET_ACCESS_KEY=<secret>
AWS_CA_BUNDLE=artifacts/magicbox-local-ca.crt \
  aws --endpoint-url https://magicbox.local:9100 --region us-east-1 s3 ls
```

Rotate keys: change them with `make vault-edit`, run `make deploy` (applied at every start; data kept), update clients.

Recreate from scratch (deletes every bucket and object):

- [ ] Portainer, environment `homelab`: delete the stack `rustfs`.
- [ ] `magicbox$ sudo docker volume rm rustfs_data`
- [ ] `controller$ make deploy`, then the checks above.

4.5 Docker registry

Images: `magicbox.local:5000/<name>:<tag>`, user `magicbox`, password `vault_registry_password`. MagicBox's own Docker engine (and the Jenkins agent) already trusts the registry; app pipelines push there and Portainer pulls from there.

- [ ] Portainer > Registries lists `magicbox` (`magicbox.local:5000`).

[ ] `https://magicbox.local/registry/` opens the registry UI; log in as `magicbox` / `vault_registry_password`; images and tags are listed.

[ ] Each LAN client that pushes or pulls trusts the CA for the registry (section 6 for the system store, plus on Linux):

```
client$ sudo mkdir -p /etc/docker/certs.d/magicbox.local:5000
client$ sudo cp magicbox-local-ca.crt /etc/docker/certs.d/magicbox.local:5000/ca.crt
```

- [ ] Docker Desktop (macOS/Windows): CA installed as in section 6, then restart Docker Desktop.
- [ ] Push and pull work:

```
client$ docker login magicbox.local:5000
client$ docker pull hello-world && docker tag hello-world magicbox.local:5000/hello-world:test
client$ docker push magicbox.local:5000/hello-world:test
client$ curl --cacert magicbox-local-ca.crt -u magicbox https://magicbox.local:5000/v2/_catalog
```

Rotate the password: `make vault-edit`, `make deploy` (rehashes the password file and updates the Jenkins credential `registry`), then `docker login` again on the clients.

Free disk space after deleting tags:

```
magicbox$ sudo docker exec registry registry garbage-collect /etc/distribution/config.yml
```

5. Verification

```
controller$ make deploy EXTRA="--tags verify"
controller$ uv run tools/idempotency_check.py playbooks/site.yml --vault-password-file <file>
```

- [ ] The `--tags verify` run passes.
- [ ] `idempotency_check.py` ends with "Idempotency check passed". Keep the password files outside the repo with mode 0600; delete them afterwards.
- [ ] From another LAN computer: `ping magicbox.local` works.
- [ ] Browser (CA trusted, section 6): `https://magicbox.local:9443/` opens with no warning; log in as `admin` with the vault password.
- [ ] `sudo reboot` on MagicBox: the page comes back, settings kept.
- [ ] `magicbox$ sudo ss -ltn | grep -E ':(9000|2375|2376) '` prints nothing.
- [ ] Portainer > Environments shows `homelab`; Stacks shows `rabbitmq`, `postgresql`, `ollama`, `observability`, `rustfs`, `registry`, `jenkins`, `kong`.
- [ ] `make status` reports every image and container as OK.

Services (from the LAN):

Service	Address	Gateway	Login
Portainer	https://magicbox.local:9443/	https://magicbox.local/portainer/	admin / vault_portainer_admin_password
Grafana	https://magicbox.local:3000/grafana/	https://magicbox.local/grafana/	admin / vault_grafana_admin_password
RabbitMQ UI	http://magicbox.local:15672/rabbitmq/	https://magicbox.local/rabbitmq/	magicbox / vault_rabbitmq_password
RabbitMQ AMQP	amqp://magicbox.local:5672	-	same as UI
RustFS console	https://magicbox.local:9101/rustfs/console/	https://magicbox.local/rustfs (redirect)	vault_rustfs_access_key / vault_rustfs_secret_key
RustFS S3 API	https://magicbox.local:9100 (path-style, us-east-1)	-	same keys as the console
PostgreSQL	magicbox.local:5432, database magicbox	-	magicbox / vault_postgresql_password

Service	Address	Gateway	Login
Open WebUI	https://magicbox.local:8444/	https://magicbox.local/openwebui (redirect)	admin@magicbox.local / vault_openwebui_admin_password
Ollama API	http://magicbox.local:11434/	https://magicbox.local/ollama/	direct: none (LAN only); gateway: header apikey
Jenkins	https://magicbox.local:8443/jenkins/	https://magicbox.local/jenkins/	admin / vault_jenkins_admin_password
Docker registry	https://magicbox.local:5000/v2/ (docker login magicbox.local:5000)	https://magicbox.local/registry/ (UI)	magicbox / vault_registry_password
Tester App (4.3)	-	https://magicbox.local/tester-app	none

- [ ] Grafana > Explore > Loki: `{container="postgresql"}` shows logs.

6. TLS trust (each client, once)

- [ ] Check the fingerprints match:

```
controller$ openssl x509 -in artifacts/magicbox-local-ca.crt -noout -fingerprint -sha256
magicbox$ sudo openssl x509 -in /opt/magicbox/portainer/certs/ca.crt -noout -fingerprint
-sha256
```

- [ ] Copy `magicbox-local-ca.crt` to the client and install it:

Client	Command / steps
macOS	sudo security add-trusted-cert -d -r trustRoot -k /Library/Keychains/System.keychain magicbox-local-ca.crt
Windows (admin PowerShell)	Import-Certificate -FilePath magicbox-local-ca.crt -CertStoreLocation Cert:\LocalMachine\Root
Ubuntu/Debian	sudo cp magicbox-local-ca.crt /usr/local/share/ca-certificates/ && sudo update-ca-certificates
Firefox	Settings > Certificates > View Certificates > Authorities > Import > trust for websites
iOS	Install the profile, then General > About > Certificate Trust Settings > enable
Android	Settings > Security > Encryption & credentials > Install a certificate > CA certificate

- [ ] Restart the browser.

The CA only covers `magicbox.local` and `magicbox`. The server certificate is renewed automatically by any playbook run within 30 days of expiry.

6.1 If a client cannot resolve magicbox.local

- [ ] Client on the same LAN subnet (not guest Wi-Fi, not VPN, not another VLAN).
- [ ] macOS, iOS, Windows 10/11: built in. Android 12+: most browsers.
- [ ] Linux: `sudo apt install avahi-daemon libnss-mdns`.
- [ ] On MagicBox: `systemctl status avahi-daemon` is active.

7. Troubleshooting

Symptom	Fix
"SSH connection originates outside"	Controller uses IPv6: keep ansible_ssh_extra_args: "-4"; ssh -4 by hand
"Timeout waiting privilege escalation prompt"	Ubuntu 26.04 (sudo-rs): set ansible_become_exe: sudo.ws
"Unsupported OS"	Install Ubuntu Server 24.04 LTS
magicbox_lan_interface assertion	Use a name listed in the error
Admin password assertion	Create vault.yml; run with --ask-vault-pass
"ufw is inactive"	Add -e magicbox_confirm_destructive=true
Controller "outside" the LAN	Fix magicbox_lan_cidr or run from the LAN
Conflicting Docker packages	Check nothing uses them, then add the confirmation flag
Docker key fingerprint mismatch	Stop and verify Docker's key out of band
magicbox.local not resolving	systemctl status avahi-daemon; see 6.1
No network over Wi-Fi	Check name/password in /etc/netplan/60-wifi.yaml; sudo netplan --debug apply
Name became magicbox-2.local	Rename the other magicbox device; restart avahi-daemon
Browser: ERR_ADDRESS_UNREACHABLE on magicbox.local	Client cached an IPv6 address: make deploy, wait 2 minutes or reconnect the client to the network
Certificate warning	Install the CA (section 6); restart the browser
Connection refused	sudo docker ps -a; sudo docker logs portainer; sudo ufw status
Vault password rejected at login	Portainer was already initialised: reset (8.4)
stacks: "Portainer rejected the vault password"	Password changed in the UI: update vault_portainer_admin_password
stacks: environment not found	Run make deploy first
Jenkins agent offline	sudo docker logs jenkins-agent; sudo docker logs jenkins (SSH errors)
Jenkins login fails	Password changed in the UI is reset at start: use vault_jenkins_admin_password
Kong: "ufw does not allow TCP port"	Run make deploy without --tags
App build: Deploy fails with HTTP 401	vault_jenkins_portainer_token missing or expired: 4.1, then make deploy
App URL returns 404	App not in the apps registry or make deploy not run
App build: "not in the MagicBox apps registry"	Add it to inventory/group_vars/magicbox/apps.yml, make deploy
App URL returns 502	App not deployed yet or wrong port: Jenkins build log; sudo docker ps
Gateway route returns 502	Service not running: make status, then make deploy
Gateway /ollama/ returns 401	Send header apikey: <vault_kong_ollama_api_key>
Old bookmark (:3000/, :15672/, :8443/) shows 404	Add /grafana/, /rabbitmq/, /jenkins/ to the URL
make deploy waits a long time on stacks	make status (MISSING image: sudo docker pull <image>)
Image pull stuck (Ollama)	sudo docker pull ollama/ollama:0.40.12 on MagicBox; df -h /var/lib
stacks: wait for containers times out	The error lists container states and logs; slow pull: re-run make deploy
Stack service rejects the vault password	Password was set at first start; change it in the service, then in the vault
docker push: "x509: certificate signed by unknown authority"	Client does not trust the CA for magicbox.local:5000: 4.5
docker push: "no basic auth credentials" / 401	docker login magicbox.local:5000 with vault_registry_password

8. Recovery

8.1 Repair drift

```
controller$ make deploy
```

8.2 Update packages

```
controller$ make update
controller$ make update EXTRA="-e os_update_reboot=true"   # if a reboot is reported
```

- [ ] Run monthly. The reboot stops Portainer and the stacks for a few minutes.

8.3 Firewall lockout

At the console: `sudo ufw disable`, fix `magicbox_lan_cidr` (IPv4 subnet), then redeploy with `-e magicbox_confirm_destructive=true`.

8.4 Lost admin password

```
magicbox$ sudo docker stop portainer
magicbox$ sudo docker run --rm -v portainer_data:/data portainer/helper-reset-password
magicbox$ sudo docker start portainer
```

8.5 Backup and restore

```
magicbox$ sudo docker stop portainer
magicbox$ sudo tar -C / -czf /root/magicbox-backup-$(date +%F).tgz opt/magicbox
var/lib/docker/volumes/portainer_data
magicbox$ sudo docker start portainer
```

Stack data lives in the volumes `postgresql_data`, `rabbitmq_data`, `grafana_data`, `loki_data`, `ollama_data`, `open_webui_data` (chats, users), `rustfs_data` (buckets), `registry_data` (images) and `jenkins_home` under `/var/lib/docker/volumes/`. Stop the stack in Portainer before copying a volume. For PostgreSQL, prefer a logical dump:

```
magicbox$ sudo docker exec postgresql pg_dumpall -U magicbox | gzip > magicbox-pg-$(date
+%F).sql.gz
```

The archive contains the CA key: store it encrypted, off the box. To restore, reinstall (sections 1-4), stop Portainer, run `sudo tar -C / -xzf <archive>`, and start Portainer.

8.6 Rotate the CA

Delete `/opt/magicbox/portainer/certs/` on MagicBox, run `make deploy`, and install the new CA on every client (also in `/etc/docker/certs.d/` for the registry, 4.5). Remove the old CA from the clients.